

```
console.log(Number.isSafeInteger(2323423423434343)); //
false
```

Installing Node.js 4.1.1 in Ubuntu

It is important if you have migrated to the latest *Node.js* environment to try ES6 features. The following two steps will get you to the latest version. If yours is not an Ubuntu environment, please refer to the *Node.js* home page.

```
curl -sL https://deb.nodesource.com/setup_4.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Staying up-to-date

If you are interested in learning more about JavaScript, take a look at Table 2, which features a few influential people you should follow.

Next month we will learn about the *for..of* iterator which

References

- [1] 'io.js and Node.js Have United and That's a Good Thing'-- <http://thenewstack.io/io-js-and-node-js-have-united-and-thats-a-good-thing/>
- [2] Detailed list of JS environments' support: <http://kangax.github.io/compat-table/es6/>
- [3] Try ES6 features online <http://www.es6fiddle.net/>

Table 2

Twitter handle	Name	Known for
@rauschma	Axel Rauschmayer	Rauschmayer is the author of the book, 'ExploringJS' and blogs at @zality
@BrendanEich	Brendan Eich	Invented JavaScript and co-founded Mozilla
@paul_irish	Paul Irish	Core skills in Chrome Dev-Tools and performance
@addyosmani	Addy Osmani	Engineer at Google working on Chrome and Polymer

can be used on strings and collection data structures like arrays. We will also learn about a shorthand notation called arrow functions. 

By: Janardan Revuru

The author works at HP EG India R&D, Bengaluru. He is passionate about JavaScript and related Web frameworks. He can be reached at janardan.revuru@gmail.com

Continued from page 64...

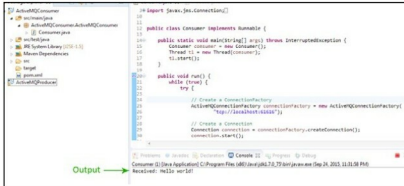


Figure 7: Consumer consuming the message

```
Destination destination = session.createQueue("TEST");

// Create a MessageProducer from the Session to
the Topic or Queue
MessageProducer producer = session.
createProducer(destination);
producer.setDeliveryMode(DeliveryMode.PERSISTENT);

// Create a messages
String text = "Hello world!";
TextMessage message = session.
createTextMessage(text);
producer.send(message);
```

```
// Clean up
session.close();
connection.close();
} catch (Exception e) {
    System.out.println("Caught: " + e);
    e.printStackTrace();
}
}
```

In the above example, the delivery mode is set to PERSISTENT, which means that even if the ActiveMQ server goes down, the *Producer* data persists.

Now, run the *Consumer* application. The *Consumer* will keep checking whether there is any message to consume from ActiveMQ.

Next, run the *Producer* application to send messages to the ActiveMQ. Each run of the *Producer* will send a 'Hello World!' message to ActiveMQ and, in turn, the *Consumer* will consume the message from ActiveMQ. 

By: Sumeet Patil

The author is a middleware expert, open source enthusiast and likes to explore new technologies. He has worked on ActiveMQ, Servicemix, Apache Camel, SAP/ABAP, SAP UI5, Gateway etc., Java/J2EE, Android native/ hybrid, HTML5, NodeJS, PHP and more. He can be reached at sumeetp1991@gmail.com